

HotelOS MVP - 3 Day Execution Plan

HotelOS MVP

Secure Hotel Reservation Management API

MVP Scope + Technical Architecture + Exact 3-Day Execution Plan

MVP rule: reduce the number of business features, not the quality or completeness of the features that are selected. Every selected feature is implemented end-to-end through Domain, Application, Infrastructure, API, authorization, persistence, validation, error handling, tests and Swagger verification.

1. MVP Goal and Release Boundary

The first release must already be a meaningful hotel application. An authorized hotel employee can log in, manage room inventory, manage guests, search date-based availability, create a correctly priced reservation, assign a physical room, search/cancel reservations and view the core hotel report.



MVP business features - exactly three

1. **Identity & Access Management**
2. **Room Inventory Management**
3. **Reservation Core** - includes Guest, Availability, Reservation, Room Assignment, Cancellation and the core Hotel Report because they form one reservation workflow.

Explicitly Post-MVP

The following remain in the full HotelOS product baseline but are **not partially implemented** in this three-day release: full Organization/Property management, Buildings/Wings/Floors/Amenities, Room Blocks, advanced Rate Plans and nightly pricing, taxes/fees/promotions, ReservationNight snapshots, Check-in/Check-out/NoShow, Folio/Payments/Refunds/Approvals, Housekeeping, Maintenance, Services/Add-ons, Customer Portal, Notifications, Online Payment Gateway, Booking.com/external channels, advanced dashboards/reports, documents and localization.

2. Feature 1 - Identity & Access Management

This feature is complete enough to secure the entire MVP. Authentication is not hard-coded and authorization is not based only on role names.

Capabilities

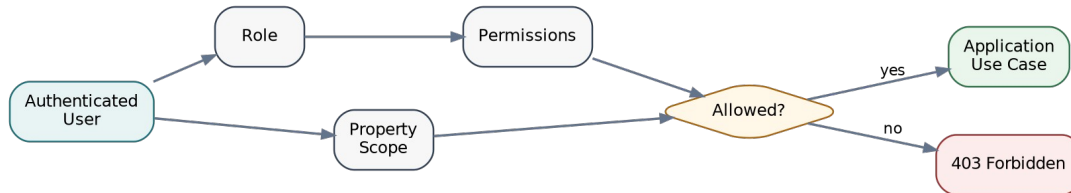
- ASP.NET Core Identity user store.
- Login with JWT access token.
- Refresh-token lifecycle.
- Logout/revoke refresh token.
- Get current authenticated user.
- Create/list/manage staff user accounts needed by the MVP.
- Roles.
- Stable permission catalogue.
- Role-to-permission assignments.
- User-to-role assignments.
- Property access scope foundation.
- Policy-based authorization.
- Seed one Organization, one Property, Admin role/user and Receptionist role/user.
- Security-sensitive role/permission changes are auditable.

MVP permission catalogue

Area	Permissions
Rooms	Rooms.View, Rooms.Manage

Area	Permissions
Guests	Guests.View, Guests.Manage
Reservations	Reservations.View, Reservations.Create, Reservations.AssignRoom, Reservations.Cancel
Reports	Reports.View
Administration	Users.Manage, Roles.Manage

Authorization decision



Authentication/API contract

Method	Route	Purpose
POST	/api/auth/login	Validate credentials and return access/refresh tokens.
POST	/api/auth/refresh	Rotate/renew token pair.
POST	/api/auth/logout	Revoke active refresh token/session.
GET	/api/auth/me	Return current user, roles, permissions and active property context.
POST	/api/users	Admin creates staff account.
GET	/api/users	Scoped/paginated staff user list.
PUT	/api/users/{id}/roles	Assign user roles.
GET	/api/roles	List roles and permission assignments.
POST	/api/roles	Create role.
PUT	/api/roles/{id}/permissions	Replace/update role permissions.

Identity model

- ApplicationUser remains authentication infrastructure.
- Role, Permission, RolePermission, UserPropertyAccess represent access control.
- Organization/Property are seeded reference data in the MVP; they are not a management feature yet.
- Tokens/secrets/passwords are never logged.

3. Feature 2 - Room Inventory Management

The MVP has a real room inventory model instead of storing reservation data inside a Room record.

RoomType

Fields:

- Id
- PropertyId
- Code
- Name
- Description
- Capacity
- BasePricePerNight
- IsActive

Rules:

- Code is unique inside the Property.
- Base price is positive.
- Capacity is positive.
- Inactive RoomType cannot be used for a new reservation.

- Deactivation preserves historical references.

Room

Fields:

- Id
- PropertyId
- RoomTypeId
- RoomNumber
- OperationalStatus = InService | OutOfService
- IsActive

Rules:

- (PropertyId, RoomNumber) is unique in SQL Server.
- Room must reference an active/valid RoomType when created.
- Out-of-service or inactive rooms do not contribute to sellable inventory.
- Room does **not** contain IsReserved; availability is derived by date.
- Transactional/history references prevent destructive deletion; DELETE semantics deactivate master data.

Room inventory API

Method	Route	Purpose
POST	/api/room-types	Create RoomType.
GET	/api/room-types	Paginated/filterable list.
GET	/api/room-types/{id}	RoomType details.
PUT	/api/room-types/{id}	Update master data.
DELETE	/api/room-types/{id}	Deactivate.
POST	/api/rooms	Create Room.
GET	/api/rooms	Paginate/filter/sort by type/status.
GET	/api/rooms/{id}	Room details.
PUT	/api/rooms/{id}	Update allowed data.
PATCH	/api/rooms/{id}/status	Change operational status.
DELETE	/api/rooms/{id}	Deactivate.

4. Feature 3 - Reservation Core

This is the business heart of the MVP. Guest, availability, pricing, reservation, physical room assignment, cancellation and the core report are implemented as one coherent workflow.

Guest sub-capability

Guest fields:

- Id
- OrganizationId
- FullName
- Phone
- Email
- CreatedAtUtc

Operations:

- Create Guest.
- Get Guest.
- Update Guest.
- Search Guest by name/phone/email.
- Pagination, filtering and organization scoping.

MVP pricing rule

Advanced pricing is a Post-MVP feature. For this release the reservation price source is `RoomType.BasePricePerNight` and the business rule is complete and explicit:

- $\text{NumberOfNights} = \text{CheckOutDate} - \text{CheckInDate}$.
- $\text{SubTotal} = \text{NumberOfNights} * \text{BasePricePerNight}$.
- Stay longer than 7 nights receives a 10% automatic discount.
- $\text{TotalAmount} = \text{SubTotal} - \text{DiscountAmount}$.
- The reservation persists `PricePerNight`, subtotal, discount and total as a **booking-time snapshot**, therefore later changes to `RoomType` price do not rewrite old bookings.
- Use decimal for money and preserve `CurrencyCode` from the seeded `Property`.

Reservation

Fields:

- Id
- ReservationNumber
- OrganizationId
- PropertyId
- GuestId
- RoomTypeId
- CheckInDate
- CheckOutDate
- NumberOfNights
- PricePerNight
- SubTotal
- DiscountPercentage
- DiscountAmount
- TotalAmount
- CurrencyCode
- Status = Confirmed | Cancelled
- CreatedAtUtc, CreatedBy
- CancelledAtUtc, CancelledBy, CancellationReason

The MVP deliberately has no half-implemented stay lifecycle. Check-in/out/no-show are Post-MVP features. A confirmed reservation may be cancelled; otherwise its booking terms are treated as confirmed data. Changes to stay dates/type are handled by cancel + rebook in MVP and become a dedicated modification workflow Post-MVP.

RoomAssignment

Fields:

- Id
- ReservationId
- RoomId
- AssignedAtUtc
- AssignedBy

Rules:

- Room belongs to the same `Property`.
- Room belongs to the booked `RoomType`.
- Room is active and `InService`.
- No active overlapping assignment exists for the same physical room.
- Cancelled reservations do not block availability.

Availability model

There are two related constraints:

1. **RoomType inventory:** confirmed reservations consume one sellable unit of the booked `RoomType` during their stay even before a physical Room is assigned.

2. Physical assignment: one physical Room cannot have overlapping active assignments.

For a requested RoomType/date range:

Sellable inventory = Active + InService Rooms of RoomType

Reserved inventory = overlapping Confirmed Reservations of RoomType

Available count = Sellable inventory - Reserved inventory

Date overlap rule:

Existing.CheckInDate < Requested.CheckOutDate

AND

Existing.CheckOutDate > Requested.CheckInDate

Concurrency invariant

The MVP must prevent two simultaneous requests from confirming the last available unit. Reservation creation executes the availability check + insert inside a SQL Server transaction using a database-safe concurrency strategy (planned baseline: Serializable isolation for this MVP). The losing request returns a conflict response rather than creating an overbooking.

Room assignment uses the same principle to prevent two concurrent assignments of the same physical room.

Reservation API

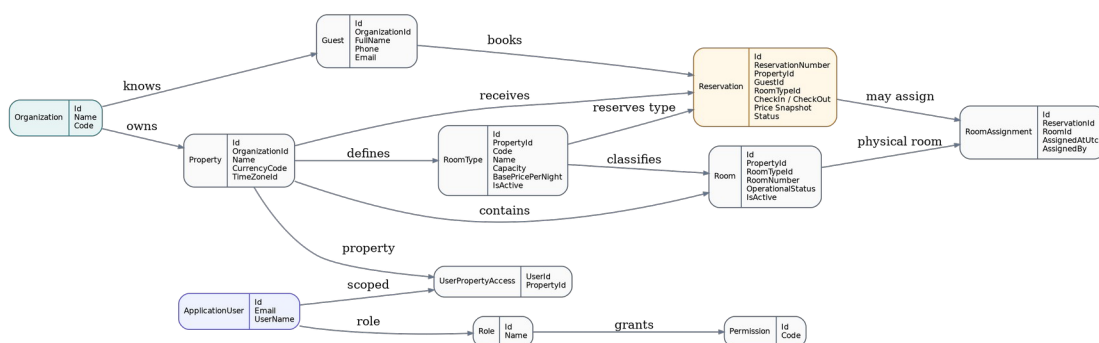
Method	Route	Purpose
POST	/api/guests	Create guest.
GET	/api/guests/{id}	Get guest.
GET	/api/guests?search=&page=&pageSize=	Search/paginate guests.
PUT	/api/guests/{id}	Update guest.
GET	/api/availability? checkIn=&checkOut=&roomTypeId=	Return room-type sellable count and current MVP price.
POST	/api/reservations	Confirm reservation after validation, availability and price calculation.
GET	/api/reservations/{id}	Detailed reservation.
GET	/api/reservations?...filters	Search/filter/sort/paginate.
POST	/api/reservations/{id}/assign-room	Assign compatible physical room.
POST	/api/reservations/{id}/cancel	Cancel with reason and audit context.
GET	/api/reports/hotel?date=	Core inventory/reservation report for selected date.

Core report

Returns:

- Total Rooms.
- Active/InService Rooms.
- Reserved Rooms/Inventory for selected date.
- Available Rooms for selected date.
- Total Confirmed Reservations.
- Total Cancelled Reservations.
- Confirmed Booking Value (sum TotalAmount for non-cancelled bookings). This is booking value, not cash revenue; financial revenue semantics arrive with the Finance feature Post-MVP.

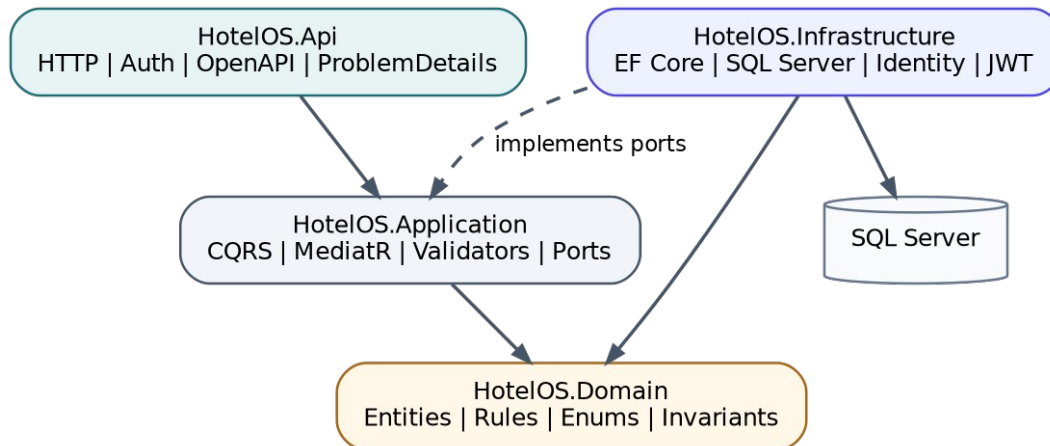
5. MVP Data Model / ERD



Key SQL Server constraints/indexes

- Unique Organization.Code.
- Unique Property code in Organization.
- Unique (PropertyId, RoomType.Code).
- Unique (PropertyId, Room.RoomNumber).
- Unique Reservation.ReservationNumber.
- Index Reservation (PropertyId, RoomTypeId, Status, CheckInDate, CheckOutDate) for availability.
- Index Room (PropertyId, RoomTypeId, OperationalStatus, IsActive).
- Index Guest (OrganizationId, Phone) and search-supporting indexes after query review.
- Monetary fields have explicit precision.
- Explicit FK delete behavior; transactional reservations are never cascade-deleted by deleting master data.

6. Technical Architecture



Solution structure

```

HotelOS/
├── HotelOS.sln
├── src/
│   ├── HotelOS.Domain/
│   ├── HotelOS.Application/
│   ├── HotelOS.Infrastructure/
│   └── HotelOS.Api/
└── tests/
    ├── HotelOS.Domain.Tests/
    ├── HotelOS.Application.Tests/
    └── HotelOS.IntegrationTests/
  
```

Project dependency rules

- Domain depends on no application/framework project.
- Application -> Domain.
- Infrastructure -> Application + Domain.
- Api -> Application + Infrastructure; API is the composition root.
- No EF Core, ASP.NET Identity, SQL Server, JWT provider or HTTP concept inside Domain.
- CQRS is an Application-layer pattern, not a separate layer.

Application organization

Feature-oriented use cases, for example:

```

Application/
├── Auth/
├── Users/
├── RoomTypes/
├── Rooms/
├── Guests/
├── Availability/
├── Reservations/
└── Reports/
  
```

A use case folder contains only what it needs, e.g.:

```
Reservations/CreateReservation/
├── CreateReservationCommand.cs
├── CreateReservationValidator.cs
├── CreateReservationHandler.cs
└── CreateReservationResponse.cs
```

EF Core / SQL Server placement

```
Infrastructure/
├── Persistence/
│   ├── HotelDbContext.cs
│   ├── Configurations/
│   ├── Repositories-or-Queries/
│   └── Migrations/
├── Identity/
├── Authentication/
└── DependencyInjection.cs
```

- Connection string belongs to API host configuration (appsettings, user secrets, environment variables), not Domain/Application.
- AddInfrastructure(configuration) wires HotelDbContext and SQL Server.
- Entity database mapping is Fluent API with IEntityTypeConfiguration<T>.
- Do not add a generic repository simply to wrap every DbSet; define focused persistence/query boundaries only when they add value.

Cross-cutting MVP requirements

- Global exception handling + standard ProblemDetails.
- FluentValidation for request/input validation.
- Domain/application checks for DB/business rules.
- Structured logging with correlation/request ID.
- CancellationToken propagated through async handlers and EF calls.
- Audit baseline for security changes and reservation create/assign/cancel actions.
- OpenAPI/Swagger documents auth, request/response and status codes.
- Secrets are never committed.

7. Feature Delivery Definition of Done

Every selected MVP use case is only complete when all applicable items are true:

1. Requirement, actor, permission, scope and acceptance rule are explicit.
2. Domain model/rules are in the correct layer.
3. Command or Query + DTO contract exists.
4. FluentValidation covers syntactic/input validation.
5. Business/data rules are enforced in handler/domain and DB constraints where applicable.
6. EF Core configuration includes relationships, precision, indexes, uniqueness and delete behavior.
7. Handler is async, accepts CancellationToken and uses explicit transaction/concurrency handling when required.
8. Endpoint has authentication + permission enforcement.
9. Errors map to predictable ProblemDetails (400/401/403/404/409/500).
10. Unit/application/integration tests cover the critical path and key edge cases.
11. Swagger happy path and negative paths are manually verified.
12. Code builds warning-clean under the agreed rules and the commit is feature-scoped.

8. Exact 3-Day Execution Plan

Target: **approximately 22-30 focused engineering hours**. This is aggressive and assumes uninterrupted work. The scope is frozen during the three-day MVP sprint; new business features move to Post-MVP.

Day 1 - Foundation + Persistence + IAM (8-10h)

Block A - Solution foundation (1.5-2h)

- Create repo/solution.
- Create 4 source projects + 3 test projects.
- Add project references and build rules.
- Create feature-oriented Application skeleton.
- Create layer DependencyInjection extension classes.
- Build cleanly.

Block B - API cross-cutting foundation (1-1.5h)

- OpenAPI/Swagger.
- Global exception handler + ProblemDetails mapping.
- Structured logging.
- Correlation/request context.
- Base API conventions.

Block C - SQL Server + EF Core (1.5-2h)

- Connection configuration.
- HotelDbContext.
- Seeded Organization/Property reference data strategy.
- First EF configurations/migration.
- Database create/update verification.
- DB health check.

Block D - IAM complete MVP slice (4-5h)

- ApplicationUser + Identity schema.
- Refresh token/session entity/strategy.
- Permission catalogue.
- RolePermission + UserPropertyAccess.
- Seed Admin/Receptionist and roles/permissions.
- Login, refresh, logout, me.
- Permission authorization policy/handler.
- User and role administration endpoints required by MVP.
- Auth/authorization integration tests.

Day 1 exit gate

- Solution builds.
- DB migrates from clean state.
- Swagger runs.
- Admin and Receptionist can log in.
- Refresh/revoke works.
- Protected endpoint rejects unauthenticated request with 401.
- Permission mismatch returns 403.
- Property scope foundation is visible in current-user context.

Day 2 - Room Inventory + Guest + Availability (7-9h)**Block A - RoomType (1.5h)**

- Domain entity/rules.
- EF configuration and unique index.
- CQRS create/get/list/update/deactivate.
- Validation.
- Authorization.
- Tests + Swagger.

Block B - Room (1.5-2h)

- Domain entity/status.
- EF relationship/unique (PropertyId, RoomNumber).
- CQRS create/get/list/update/status/deactivate.
- Filtering/sorting/pagination.
- Authorization.
- Tests.

Block C - Guest (1.5h)

- Guest entity/configuration.
- Create/get/update/search.
- Search + pagination + scoped projection.

- Validation/permissions/tests.

Block D - Availability (2.5-4h)

- Define availability query contract.
- Implement date overlap query.
- Count active/in-service RoomType inventory.
- Subtract overlapping confirmed reservations.
- Return price from RoomType snapshot source.
- Add/query indexes.
- Test boundaries: adjacent stays are not overlap; true overlaps are rejected/count correctly.
- Test cancelled reservations do not consume inventory.

Day 2 exit gate

- Admin can create RoomType and rooms.
- Duplicate room number is DB/application rejected.
- Receptionist can view but unauthorized management is forbidden as configured.
- Guest can be created/searched.
- Availability query returns correct count for empty and occupied date ranges.
- Out-of-service rooms are excluded.

Day 3 - Reservation Core + Concurrency + Report + Hardening (8-11h)

Block A - Reservation domain + persistence (1.5-2h)

- Reservation entity, status, pricing snapshot.
- RoomAssignment entity.
- Configurations/indexes.
- Reservation number generation strategy.
- Migration review.

Block B - Create Reservation (2-3h)

- Command/response/validator.
- Guest/RoomType/date checks.
- Number of nights.
- Base price snapshot.
- Stay longer than 7 nights => 10% discount.
- Availability check.
- Serializable transaction/concurrency handling.
- Audit entry.
- 201 response + ProblemDetails conflicts.

Block C - Read/Assign/Cancel (2-2.5h)

- Get Reservation projection.
- Search/filter/sort/paginate.
- Assign physical Room with type/property/status/overlap checks.
- Cancel with reason, actor/time and audit.
- Verify cancelled booking restores sellable inventory.

Block D - Report + final quality (2.5-3.5h)

- Core hotel report query.
- Domain/application tests for pricing/discount.
- Real SQL integration tests for availability and constraints.
- Concurrent booking test: final unit -> one success, one conflict.
- Authorization tests.
- Swagger examples and final manual demo.
- Clean seed/demo data.
- README: architecture, setup, migration/run commands and demo flow.
- Final dependency/code review.

Day 3 exit gate / MVP release

- Full demo path works from Swagger.
- No cross-property/unauthorized access through tested paths.
- Overbooking is prevented under concurrency test.
- Reservation pricing is snapshotted and old reservations do not change when RoomType base price changes.
- Cancellation releases inventory.
- Core report matches DB state.
- Critical tests pass.
- Clean migration from empty DB works.
- Tag/release candidate: v0.1.0-mvp.

9. Exact Implementation Backlog and Suggested Commits

Order	ID	Task	Suggested commit
1	FND-01	Solution/projects/references/build rules	chore: initialize clean architecture solution
2	FND-02	DI/composition root/OpenAPI	chore: configure application and infrastructure services
3	FND-03	ProblemDetails/logging/correlation	feat: add api error handling and request observability
4	DB-01	SQL Server/DbContext/migrations/health	feat: add ef core sql server persistence foundation
5	IAM-01	Identity schema + roles/permissions seed	feat: add identity roles and permission model
6	IAM-02	JWT/refresh/login/logout/me	feat: implement jwt authentication lifecycle
7	IAM-03	Permission + property-scope authorization/admin	feat: enforce permission based authorization
8	INV-01	RoomType complete slice	feat: implement room type management
9	INV-02	Room complete slice	feat: implement room inventory management
10	GST-01	Guest CRUD/search	feat: implement guest management
11	AVL-01	Availability query + indexes/tests	feat: implement date based room availability
12	RES-01	Reservation model + create/pricing/concurrency	feat: implement reservation creation
13	RES-02	Get/search reservations	feat: implement reservation queries
14	RES-03	Physical room assignment	feat: implement reservation room assignment
15	RES-04	Cancellation	feat: implement reservation cancellation
16	RPT-01	Core hotel report	feat: add core hotel reservation report
17	QLT-01	Integration/auth/concurrency hardening/docs	test: harden mvp critical workflows

10. MVP Acceptance Demo

The MVP is accepted only if this exact demo can be performed against a clean database:

1. Run migration / start API.
2. Login as Admin.
3. Verify Admin permissions from /auth/me.
4. Create Deluxe RoomType with BasePricePerNight = 1000.
5. Create Room 101, 102, 103.
6. Create a Guest.
7. Search availability for 3 nights -> 3 available.
8. Create reservation -> Confirmed, snapshot total = 3000.
9. Search same dates -> 2 available.
10. Assign Room 101.
11. Attempt overlapping assignment to Room 101 -> 409.
12. Create >7-night reservation -> 10% discount is applied.
13. Search reservations by guest/reservation/status/date.
14. Cancel first reservation with reason.
15. Search original dates -> inventory is available again.
16. Change RoomType base price and prove old Reservation total is unchanged.
17. View core Hotel Report and reconcile it with DB state.
18. Login as Receptionist and prove allowed actions work.
19. Attempt an action without permission -> 403.
20. Run critical unit + integration + concurrency tests successfully.

11. Post-MVP Roadmap - Full Features, Not Partial Additions

After v0.1.0-mvp, expand the same architecture without rewriting the MVP.

Post-MVP sequence	Complete feature package
P1	Organization & Property Management - organizations, multiple properties, Buildings/Wings, Floors, Amenities, richer staff/property administration.
P2	Advanced Pricing - Rate Plans, nightly rates, cancellation policies, promotions/discount engine, taxes/fees, ReservationNight snapshots.
P3	Stay Management - full Pending/Confirmed/CheckedIn/CheckedOut/Cancelled/NoShow lifecycle, check-in, check-out, room readiness and room-change workflow.
P4	Finance - Folio, charges, partial payments, payment methods/providers, refunds, voids and approval workflow.
P5	Hotel Operations - Housekeeping, Maintenance, date-bounded Room Blocks, Service Catalogue and Add-ons.
P6	Customer Experience - Guest/customer account linking, customer portal, self-service booking/manage flow and documents.
P7	Communication & Integrations - notifications, email/SMS/WhatsApp adapters, online payment gateway, booking channels/Booking.com adapter, idempotency/sync workflow.
P8	Management & Platform Quality - advanced operational/financial dashboards, audit viewer/activity timeline, documents/storage, localization and remaining platform hardening.



12. Final MVP Quality Bar

The three-day goal is **not** "finish every HotelOS feature." The goal is to release the smallest business slice that is already architecturally serious and demonstrably useful.

Release statement:

HotelOS MVP is a secure, production-structured ASP.NET Core hotel reservation API in which authorized staff can manage room inventory and guests, calculate date-based availability, create correctly priced reservations with a booking-time snapshot and long-stay discount, prevent overbooking under concurrency, assign physical rooms, search/cancel reservations, and obtain the core hotel report. The implementation uses Clean Architecture, CQRS/MediatR, FluentValidation, EF Core/SQL Server, ASP.NET Core Identity, JWT, roles + permissions + scope, global ProblemDetails, structured logging, Swagger and automated tests.

Estimated focused effort: 22-30 hours across 3 aggressive development days.